

# Real-Time : Crypto Market Monitoring System

## Introduction

Les marchés financiers crypto génèrent des **flux de données massifs en temps réel** (transactions, prix, volumes).

Votre mission est de **concevoir un système de streaming de données temps réel** capable de :

- **ingérer** des flux live de marché
- **traiter** les données en continu
- **détecter** des événements importants
- **afficher** des résultats exploitables en temps réel (Dashboard)

Construire un pipeline de données temps réel complet basé sur des flux crypto issus de Binance et Coinbase (**si vous trouvez d'autres solutions en ligne équivalente vous pouvez les utiliser**).

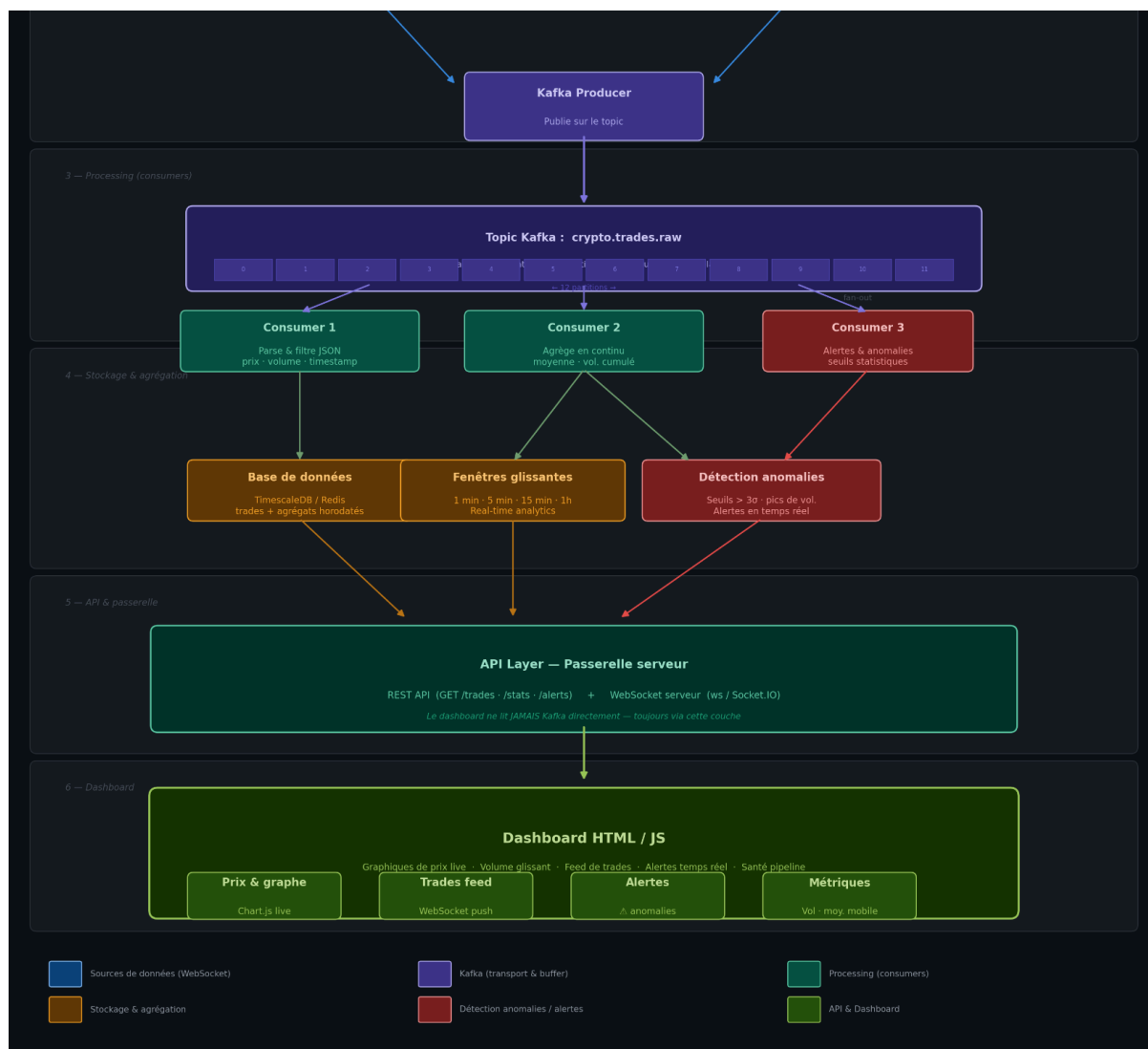
## Architecture attendue

Vous commencerez par mettre en place une phase d'ingestion de données (streaming) en vous connectant à une **API WebSocket** d'un exchange crypto, afin de recevoir en continu les transactions (trades) dès qu'elles sont générées sur le marché.

Ces données brutes devront ensuite être prises en charge dans une phase de data processing, où vous serez amenés à parser les événements au format JSON, à filtrer les informations pertinentes (par exemple **prix, volume, horodatage**) et à effectuer des premières opérations d'agrégation comme le calcul de moyennes ou de volumes cumulés.

Dans un vrai système industriel :

- les flux arrivent très vite (tick market)
- les traitements peuvent ralentir
- les consommateurs peuvent tomber
- plusieurs systèmes doivent consommer les mêmes données



Pour éviter cette problématique vous devrez utiliser Kafka car dans le projet, ce composant joue un rôle central de tampon (buffer) entre les flux entrants et les traitements. Il permet de découpler l'ingestion des données de leur traitement, ce qui évite que les deux étapes soient dépendantes en temps réel. Grâce à ce mécanisme, le système gagne en résilience : en cas de ralentissement ou de panne côté traitement, les données continuent d'être collectées sans perte immédiate.

Enfin, ce tampon rend possible la diffusion des mêmes flux vers plusieurs consommateurs en parallèle, facilitant ainsi la scalabilité et l'extension des usages sans impacter la chaîne d'ingestion.

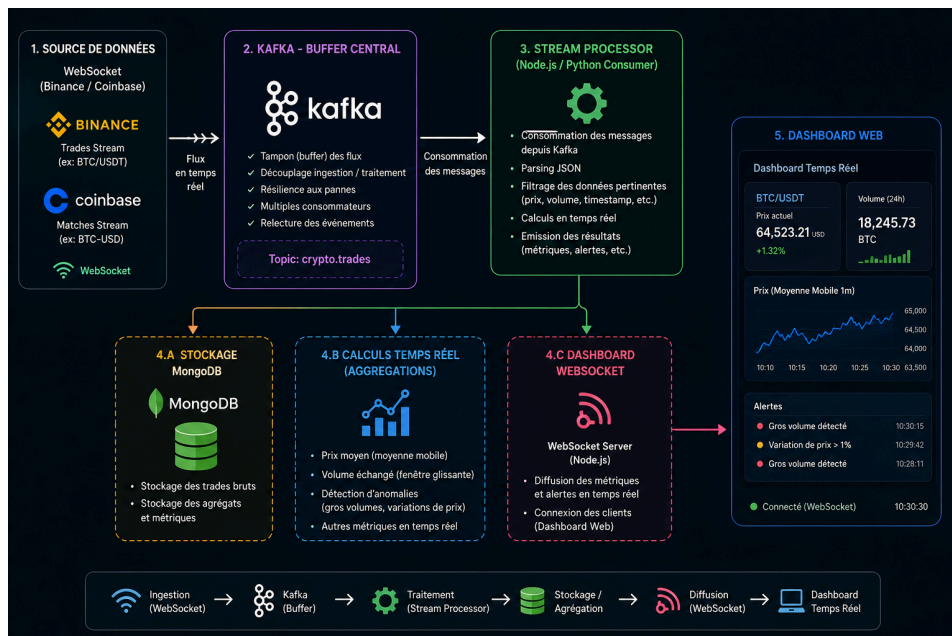
Enfin, vous développerez une couche de real-time analytics, permettant de calculer et mettre à jour en continu différentes métriques de marché telles que le prix moyen, le volume échangé sur une période glissante, la détection de transactions anormalement élevées ainsi que l'analyse des variations de prix en temps réel en exposant ses données via une api REST FULL.

## Exposition des données vers le Dashboard

Pour alimenter votre dashboard en temps réel, vous devrez mettre en place une API WebSocket côté serveur (par exemple avec la librairie ws en Node.js, ou Socket.IO). Ce composant joue le rôle de passerelle entre votre pipeline Kafka et le navigateur : le serveur s'abonne aux messages produits par vos consumers Kafka, les formate, puis les rediffuse en push à tous les clients connectés sans que le dashboard ait besoin d'interroger le serveur à intervalle régulier. A vous de choisir soit vous passez par HTTP soit par WebSocket.

### *Point d'attention une erreur fréquente à éviter*

*Il peut être tentant de faire lire le dashboard directement dans Kafka ou dans la base de données. Ce n'est pas la bonne approche. Le dashboard ne doit jamais être un consumer Kafka direct. Vous devez systématiquement passer par une couche API intermédiaire, qui agrège les données, les formate et les pousse au bon moment. C'est ce découplage qui garantit la robustesse et la scalabilité de votre architecture.*



## Données à utilisées

Vous devez utiliser au minimum une API streaming réelle :

### **Binance Stream :**

- BTC/USDT ou ETH/USDT trades
- flux continu (plusieurs événements / seconde)

### **Coinbase Stream :**

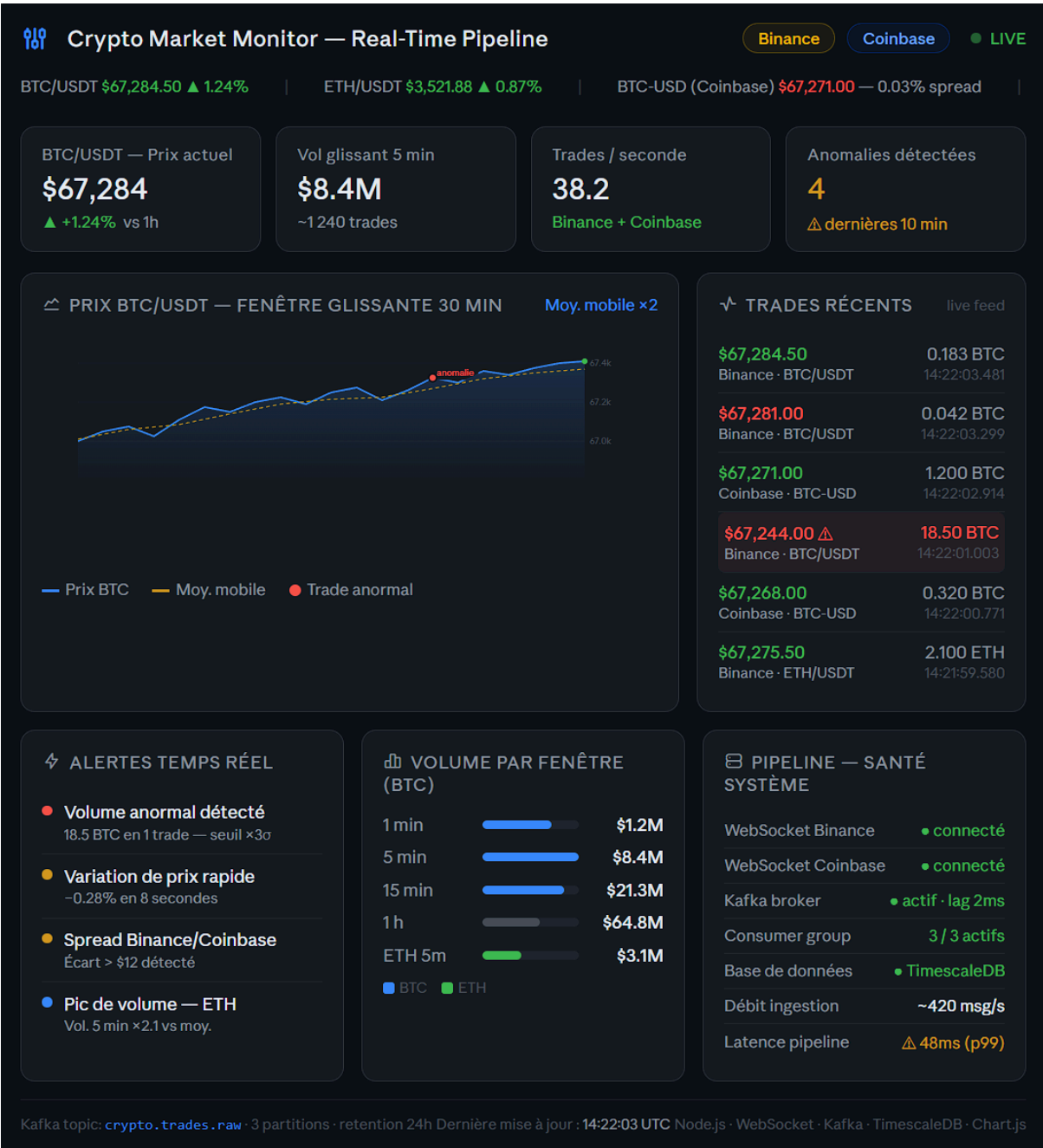
- BTC-USD matches
- flux structuré avec abonnement

# Dashboard

En complément, vous devrez concevoir un **dashboard en HTML, CSS et JavaScript** permettant d’afficher en temps réel les données issues de votre pipeline de streaming.

Ce dashboard devra être directement connecté à votre système de traitement afin de visualiser les résultats des calculs et agrégations en continu. Vous serez également attendus sur la qualité de l’interface proposée : vous devrez être force de proposition afin de concevoir une interface claire, structurée et ergonomique, capable de mettre en valeur les données temps réel et de rendre lisible l’évolution du marché.

Voila un exemple de à quoi pourra ressembler votre dashboard :



## Soutenance

**Durée** : 10 à 15 minutes par groupe de 4 minimum à 5 maximum

Lors de la soutenance, vous devrez présenter l'architecture globale de votre système en expliquant clairement le pipeline de données mis en place, depuis l'ingestion des données jusqu'au traitement en temps réel et à la restitution des résultats.

Vous réaliserez ensuite une démonstration en direct de votre projet, en montrant l'affichage des flux en temps réel ainsi que les différentes métriques calculées à partir des données traitées. Vous devrez également fournir une explication technique détaillée de votre solution, en abordant notamment l'utilisation des WebSockets / REST, les mécanismes de traitement en streaming ainsi que la logique des algorithmes implémentés.

## **Contraintes techniques**

- utilisation de WebSocket
- traitement en temps réel obligatoire
- langage libre (JS recommandé)
- pas de batch processing
- affichage live obligatoire (Dashboard)